

데이터 과학 및 분석

로지스틱 회귀분석

호서대학교 조학수

01 로지스틱회귀분석

02 실습

01 로지스틱회귀분석

01 로지스틱 회귀

■ 선형 회귀와 확률 추정

- 선형 회귀는 특성 X 를 입력으로 받아 편향과 가중치를 곱하여 추정값을 계산하는 과정

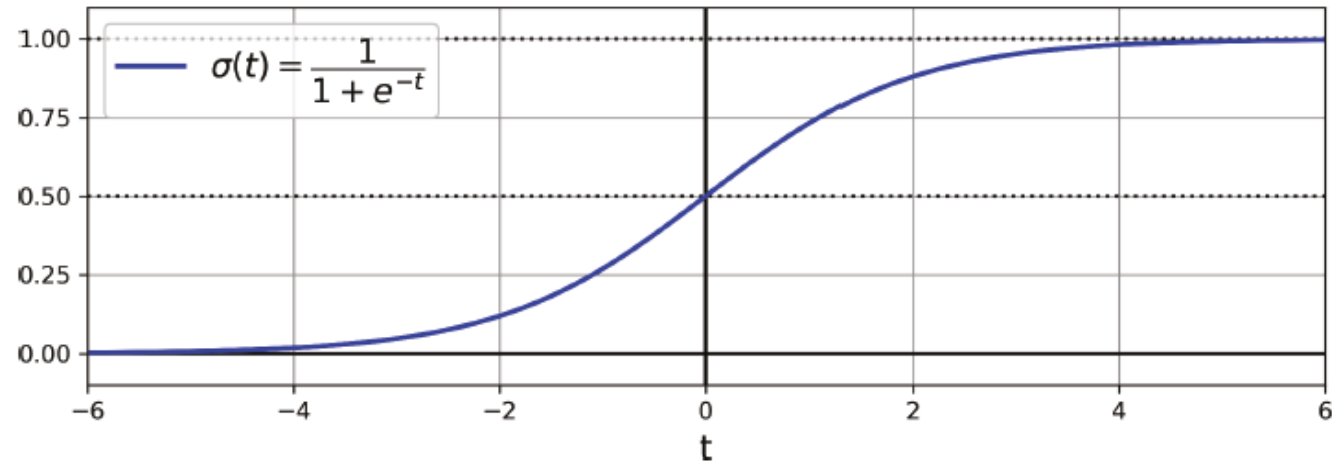
$$\hat{y} = \theta^T X$$

로지스틱 (시그모이드) 함수

- 선형회귀를 분류 문제에 사용하기는 부적합 ➔ 선형회귀의 값을 확률값으로 변환하면 어떨까?

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$

로지스틱 함수 (시그모이드 함수)



- 로지스틱 함수를 이용해 확률을 추정 (0~1 사이 값 계산)

$$\hat{p} = h_{\theta}(X) = \sigma(\theta^T X)$$

01 로지스틱 회귀

- 로지스틱 회귀 모델이 샘플 x 가 양성 클래스에 속할 확률 $\hat{p} = h_{\theta}(X)$ 를 계산하여 $\hat{y}$ 를 추정

로지스틱 회귀 모델 예측

$$\hat{y} = \begin{cases} 0, & \hat{p} < 0.5 \text{일때} \\ 1, & \hat{p} \geq 0.5 \text{일때} \end{cases}$$

01 로지스틱 회귀

■ 훈련과 비용 함수

■ 훈련의 목적

- 양성 샘플($y = 1$)에 대해서는 높은 확률을 추정하고, 음성 샘플($y = 0$)에 대해서는 낮은 확률을 추정하는 모델의 파라미터 벡터 θ 를 찾는 것
- 비용함수는 $y = 1$ 일 때 확률 $\hat{p}$ 를 1로 추정하면 비용이 0
- $y = 0$ 일 때 확률 $\hat{p}$ 를 0으로 추정하면 비용이 0

하나의 훈련 샘플에 대한 비용 함수
- log loss 사용

$$c(\theta) = \begin{cases} -\log(\hat{p}) & y = 1 \text{ 일 때} \\ -\log(1 - \hat{p}) & y = 0 \text{ 일 때} \end{cases}$$

■ 비용 함수

- 로그 손실(log loss) - 전체 훈련 세트에 대한 비용 함수는 모든 훈련 샘플의 비용을 평균

로지스틱 회귀의 비용 함수(로그 손실)

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)})]$$

$y = 1$ 일때

$y = 0$ 일때

01 로지스틱 회귀

■ 훈련과 비용 함수

■ 훈련의 목적

- 양성 샘플($y = 1$)에 대해서는 높은 확률을 추정하고 음성 샘플($y = 0$)에 대해서는 낮은 확률을 추정하는 모델의 파라미터 벡터 θ 를 찾는 것

하나의 훈련 샘플에 대한 비용 함수

$$c(\theta) = \begin{cases} -\log(\hat{p}) & y = 1 \text{일 때} \\ -\log(1 - \hat{p}) & y = 0 \text{일 때} \end{cases}$$

■ 비용 함수

- 로그 손실(log loss) - 전체 훈련 세트에 대한 비용 함수는 모든 훈련 샘플의 비용을 평균

로지스틱 회귀의 비용 함수(로그 손실)

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)})]$$

■ 비용 함수의 편도함수

식 4-18 로지스틱 비용 함수의 편도함수

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^m (\sigma(\theta^T \mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

참고자료

■ $p = \sigma(t)$ 의 미분

- $\hat{p} = h_{\theta}(X) = \sigma(\theta^T X)$
- $\frac{\partial \hat{p}}{\partial t} = \frac{\partial \sigma(t)}{\partial t} = \sigma(t)(1 - \sigma(t)) = p(1 - p)$

■ $t = \theta^T X$ 의 미분

- $\frac{\partial \theta^T x^{(i)}}{\partial \theta_j} = x_j^{(i)}$

■ 로지스틱 비용함수

- $J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)})]$

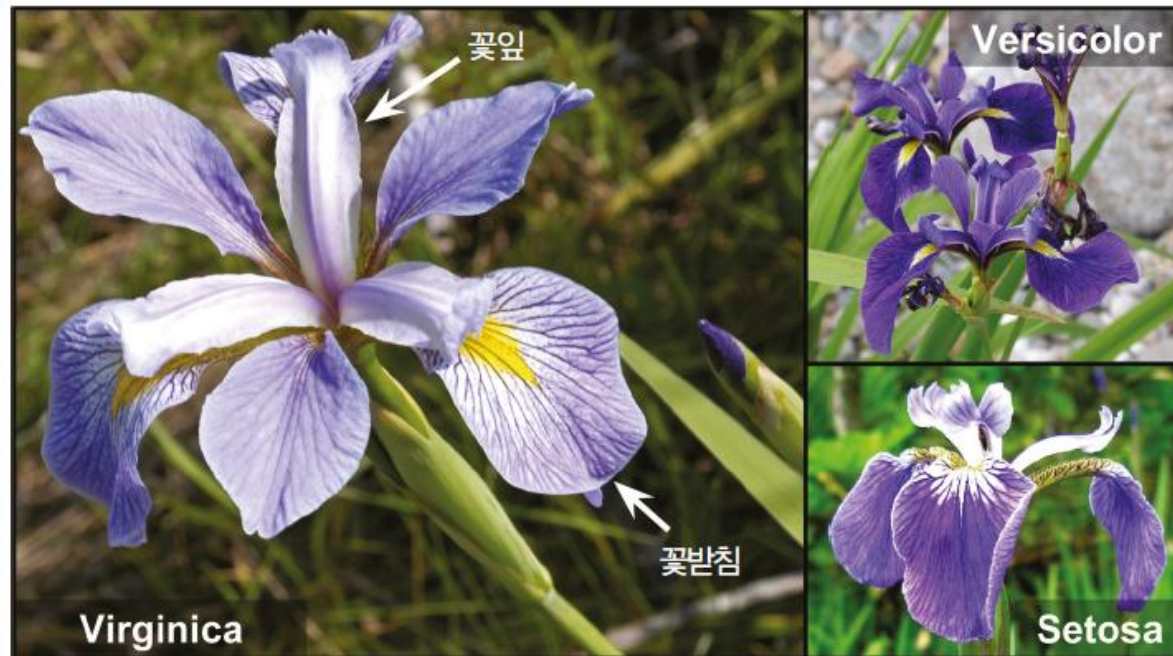
■ 로지스틱 비용 편도함수

- $\frac{\partial J(\theta)}{\partial \theta_j} = \hat{p}' \frac{y - \hat{p}}{\hat{p}(1 - \hat{p})} = \frac{1}{m} \sum_{i=1}^m (\sigma(\theta^T X^{(i)}) - y^{(i)}) x_j^{(i)}$

01 로지스틱 회귀

■ 결정 경계

- 붓꽃(Iris) 데이터셋
 - 세 개의 품종 Iris-Setosa, Iris-Versicolor, Iris-Virginica
 - 150개 꽃잎(petal)과 꽃받침(sepal)의 너비와 길이 데이터



세 종류의 붓꽃

01 로지스틱 회귀

- 꽃잎의 너비를 기반으로 Iris-Versicolor 종을 감지하는 분류기 만들기

```
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
list(iris)
```

```
➡ ['data',
   'target',
   'frame',
   'target_names',
   'DESCR',
   'feature_names',
   'filename',
   'data_module']
```

01 로지스틱 회귀

- 꽃잎의 너비를 기반으로 Iris-Versicolor 종을 감지하는 분류기 만들기

```
iris.data.head(3)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2

01 로지스틱 회귀

- 꽃잎의 너비를 기반으로 Iris-Versicolor 종을 감지하는 분류기 만들기

```
iris.target.head(3) # 샘플이 섞여 있지 않습니다.
```

↩

	target
0	0
1	0
2	0

dtype: int64

01 로지스틱 회귀

- 꽃잎의 너비를 기반으로 Iris-Versicolor 종을 감지하는 분류기 만들기

```
iris.target_names
```

```
➡ array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

01 로지스틱 회귀

- 데이터를 분할하고 훈련 세트에서 로지스틱 회귀 모델을 훈련

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

X = iris.data[["petal width (cm)"]].values
y = iris.target_names[iris.target] == 'virginica'
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train, y_train)
```



```
LogisticRegression
LogisticRegression(random_state=42)
```

01 로지스틱 회귀

- 꽃잎의 너비가 0~3cm인 꽃에 대해 모델의 추정 확률을 계산

```
X_new = np.linspace(0, 3, 1000).reshape(-1, 1)
y_proba = log_reg.predict_proba(X_new)
decision_boundary = X_new[y_proba[:, 1] >= 0.5][0, 0]

plt.figure(figsize=(8, 3))
plt.plot(X_new, y_proba[:, 0], "b--", linewidth=2,
         label="Not Iris virginica proba")
plt.plot(X_new, y_proba[:, 1], "g-", linewidth=2, label="Iris virginica proba")
plt.plot([decision_boundary, decision_boundary], [0, 1], "k:", linewidth=2,
         label="Decision boundary")

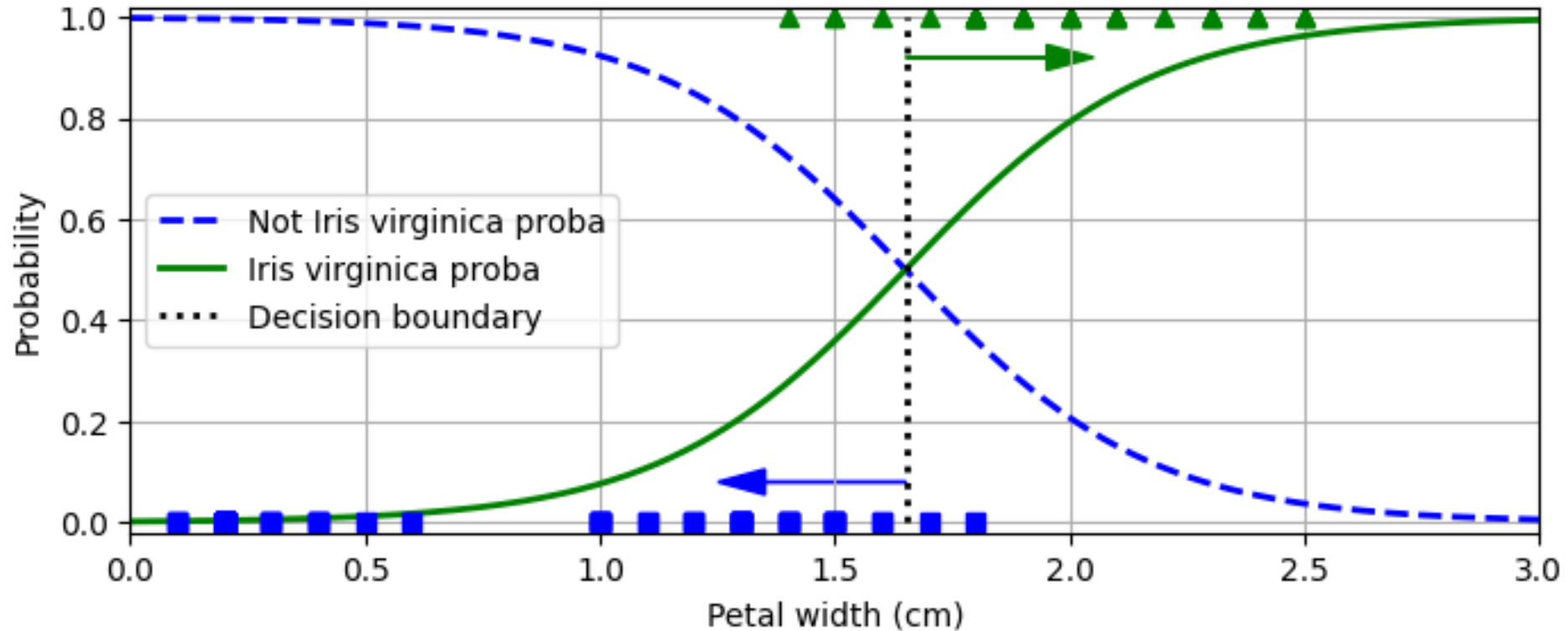
plt.arrow(x=decision_boundary, y=0.08, dx=-0.3, dy=0,
          head_width=0.05, head_length=0.1, fc="b", ec="b")
plt.arrow(x=decision_boundary, y=0.92, dx=0.3, dy=0,
          head_width=0.05, head_length=0.1, fc="g", ec="g")
plt.plot(X_train[y_train == 0], y_train[y_train == 0], "bs")
plt.plot(X_train[y_train == 1], y_train[y_train == 1], "g^")
plt.xlabel("Petal width (cm)")
plt.ylabel("Probability")
plt.legend(loc="center left")
plt.axis([0, 3, -0.02, 1.02])
plt.grid()

plt.show()
```

01 로지스틱 회귀

- 꽃잎의 너비가 0~3cm인 꽃에 대해 모델의 추정 확률을 계산

→



01 로지스틱 회귀

- predict_proba() 메서드 대신 predict() 메서드를 사용하여 클래스를 예측

```
decision_boundary
```

```
⇒ 1.6516516516516517
```

```
log_reg.predict([[1.7], [1.5]])
```

```
⇒ array([ True, False])
```

01 로지스틱 회귀

- predict_proba() 메서드 대신 predict() 메서드를 사용하여 클래스를 예측

```
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = iris.target_names[iris.target] == 'virginica'
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

log_reg = LogisticRegression(C=2, random_state=42)
log_reg.fit(X_train, y_train)

# 등고선 그래프
x0, x1 = np.meshgrid(np.linspace(2.9, 7, 500).reshape(-1, 1),
                     np.linspace(0.8, 2.7, 200).reshape(-1, 1))
X_new = np.c_[x0.ravel(), x1.ravel()] # 그림의 포인트당 하나의 샘플
y_proba = log_reg.predict_proba(X_new)
zz = y_proba[:, 1].reshape(x0.shape)

# 결정 경계
left_right = np.array([2.9, 7])
boundary = -((log_reg.coef_[0, 0] * left_right + log_reg.intercept_[0])
             / log_reg.coef_[0, 1])
```

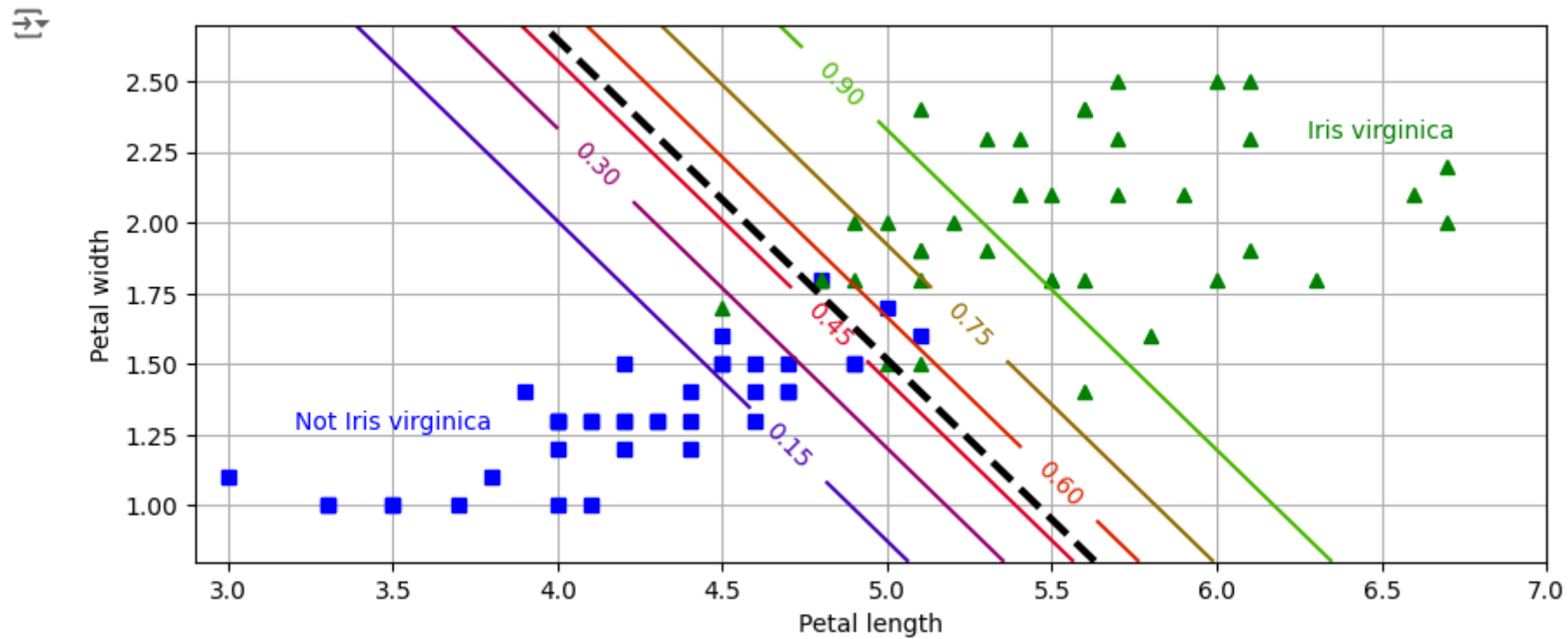
01 로지스틱 회귀

- predict_proba() 메서드 대신 predict() 메서드를 사용하여 클래스를 예측

```
plt.figure(figsize=(10, 4))
plt.plot(X_train[y_train == 0, 0], X_train[y_train == 0, 1], "bs")
plt.plot(X_train[y_train == 1, 0], X_train[y_train == 1, 1], "g^")
contour = plt.contour(x0, x1, zz, cmap=plt.cm.brg)
plt.clabel(contour, inline=1)
plt.plot(left_right, boundary, "k--", linewidth=3)
plt.text(3.5, 1.27, "Not Iris virginica", color="b", ha="center")
plt.text(6.5, 2.3, "Iris virginica", color="g", ha="center")
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.axis([2.9, 7, 0.8, 2.7])
plt.grid()
plt.show()
```

01 로지스틱 회귀

- predict_proba() 메서드 대신 predict() 메서드를 사용하여 클래스를 예측



01 로지스틱 회귀

■ 소프트맥스 회귀

- 소프트맥스 회귀(softmax regression) 또는 다항 로지스틱 회귀(multinomial logistic regression)
 - 여러 개의 이진 분류기를 훈련시켜 연결하지 않고 직접 다중 클래스를 지원하도록 일반화
 - 샘플 $\mathbf{x}$ 가 주어지면 먼저 소프트맥스 회귀 모델이 각 클래스 k 에 대한 점수 $s_k(\mathbf{x})$ 를 계산하고, 그 점수에 소프트맥스 함수 (또는 정규화된 지수 함수)를 적용하여 각 클래스의 확률을 추정

식 4-19 클래스 k 에 대한 소프트맥스 점수

$$s_k(\mathbf{x}) = (\boldsymbol{\theta}^{(k)})^T \mathbf{x}$$

식 4-20 소프트맥스 함수

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}$$

식 4-21 소프트맥스 회귀 분류기의 예측

$$\hat{y} = \underset{k}{\operatorname{argmax}} \sigma(\mathbf{s}(\mathbf{x}))_k = \underset{k}{\operatorname{argmax}} s_k(\mathbf{x}) = \underset{k}{\operatorname{argmax}} ((\boldsymbol{\theta}^{(k)})^T \mathbf{x})$$

식 4-22 크로스 엔트로피 비용 함수

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{p}_k^{(i)})$$

식 4-23 클래스 k 에 대한 크로스 엔트로피의 그레디언트 벡터

$$\nabla_{\boldsymbol{\theta}^{(k)}} J(\Theta) = \frac{1}{m} \sum_{i=1}^m (\hat{p}_k^{(i)} - y_k^{(i)}) \mathbf{x}^{(i)}$$

01 로지스틱 회귀

- 소프트맥스 회귀를 사용해 붓꽃을 세 개의 클래스로 분류
 - 사이킷런의 LogisticRegression은 클래스가 둘 이상일 때 자동으로 소프트맥스 회귀를 사용

```
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = iris["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

softmax_reg = LogisticRegression(C=30, random_state=42)
softmax_reg.fit(X_train, y_train)
```

- 꽃잎의 길이가 5cm, 너비가 2cm인 붓꽃을 발견했다고 가정하고 이 붓꽃의 품종이 무엇인지 모델에 질의

```
softmax_reg.predict([[5, 2]])
```

⇒ array([2])

```
softmax_reg.predict_proba([[5, 2]]).round(2)
```

⇒ array([[0. , 0.04, 0.96]])

01 로지스틱 회귀

- 소프트맥스 회귀 결정 경계
 - 결정 경계를 배경색으로 구분
 - 클래스 사이의 결정 경계가 모두 선형
 - Iris-Versicolor 클래스에 대한 확률은 곡선

```
from matplotlib.colors import ListedColormap

custom_cmap = ListedColormap(["#fafab0", "#9898ff", "#a0faa0"])

x0, x1 = np.meshgrid(np.linspace(0, 8, 500).reshape(-1, 1),
                     np.linspace(0, 3.5, 200).reshape(-1, 1))
X_new = np.c_[x0.ravel(), x1.ravel()]

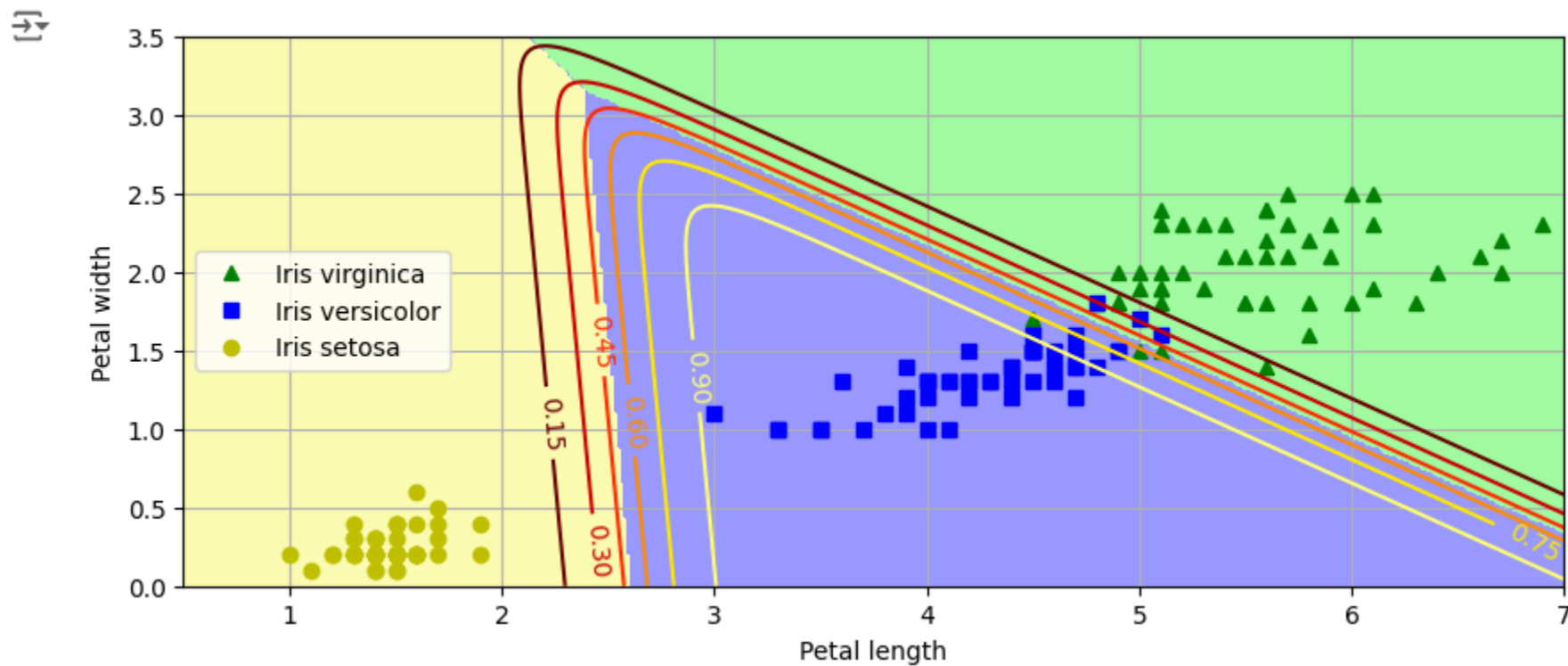
y_proba = softmax_reg.predict_proba(X_new)
y_predict = softmax_reg.predict(X_new)

zz1 = y_proba[:, 1].reshape(x0.shape)
zz = y_predict.reshape(x0.shape)
```

```
plt.figure(figsize=(10, 4))
plt.plot(X[y == 2, 0], X[y == 2, 1], "g^", label="Iris virginica")
plt.plot(X[y == 1, 0], X[y == 1, 1], "bs", label="Iris versicolor")
plt.plot(X[y == 0, 0], X[y == 0, 1], "yo", label="Iris setosa")

plt.contourf(x0, x1, zz, cmap=custom_cmap)
contour = plt.contour(x0, x1, zz1, cmap="hot")
plt.clabel(contour, inline=1)
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.legend(loc="center left")
plt.axis([0.5, 7, 0, 3.5])
plt.grid()
plt.show()
```


01 로지스틱 회귀



02 실습

■ MNIST 데이터셋

- 고등학생과 미국 인구조사국 직원들이 손으로 쓴 70,000개의 작은 숫자 이미지. 각 이미지에는 어떤 숫자를 나타내는지 레이블 되어 있음(학습용으로 매우 많이 사용됨)
- MNIST 데이터셋 내려받기

```
from sklearn.datasets import fetch_openml  
  
mnist = fetch_openml('mnist_784', as_frame=False)
```

- sklearn.datasets 패키지의 함수
 - 실전 데이터셋을 다운로드하기 위한 fetch_* 함수
 - 사이킷런에 번들로 포함된 소규모 데이터셋을 로드하기 위한 load_* 함수
 - 테스트에 유용한 가짜 데이터셋을 생성하기 위한 make_* 함수
- 사이킷런에서 읽어 들인 데이터셋들의 일반적인 디렉터리 구조
 - DESCR: 데이터셋 설명
 - data: 입력 데이터, 일반적으로 2D 넘파이 배열
 - target: 레이블, 일반적으로 1D 넘파이 배열

02 MNIST

- fetch_openml() 함수는 기본적으로 입력을 판다스 데이터프레임, 레이블을 판다스 시리즈로 반환
 - MNIST 데이터셋은 이미지이므로 데이터프레임이 잘 맞지 않음
 - 따라서 as_frame=False로 지정하여 넘파이 배열로 데이터를 받음

```
X, y = mnist.data, mnist.target
X
```

```
array([[0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]])
```

```
X.shape
```

```
(70000, 784)
```

```
y
```

```
array(['5', '0', '4', ..., '4', '5', '6'], dtype=object)
```

```
y.shape
```

```
(70000,)
```

02 MNIST

- 데이터셋에서 이미지 하나를 확인(그림 3-1)
 - 샘플의 특성 벡터를 추출해서 28×28 배열로 크기를 바꾸고 맷플롯립의 imshow() 함수를 사용
 - cmap="binary"로 지정해 0을 흰색, 255를 검은색으로 나타내는 흑백(grayscale) 컬러 맵을 사용

```
import matplotlib.pyplot as plt

def plot_digit(image_data):
    image = image_data.reshape(28, 28)
    plt.imshow(image, cmap="binary")
    plt.axis("off")

some_digit = X[0]
plot_digit(some_digit)
plt.show()
```



- 숫자 5로 보이는 그림의 실제 레이블을 확인

```
y[0]
```



```
'5'
```

02 MNIST

- MNIST 이미지 샘플



MNIST 데이터셋에서 추출한 숫자 이미지

02 MNIST

- `fetch_openml()`이 반환한 MNIST 데이터셋은 이미 훈련 세트(앞쪽 60,000개 이미지)와 테스트 세트(뒤쪽 10,000개 이미지)로 분리

```
X_train, X_test, y_train, y_test = X[:60000], X[60000:], y[:60000], y[60000:]
```

- 훈련 세트는 이미 섞여 있어서 모든 교차 검증 폴드를 비슷하게 만들
(하나의 폴드라도 특정 숫자가 누락되면 안 됨)
- 어떤 학습 알고리즘은 훈련 샘플의 순서에 민감해서 많은 비슷한 샘플이 연이어 나타나면 성능이 나빠짐
 - 데이터셋을 섞어서 이런 문제를 방지

02 MNIST

- '5-감지기'는 '5'와 '5 아님' 두 개의 클래스를 구분할 수 있는 이진 분류기(binary classifier)
 - 타겟 벡터 생성

```
y_train_5 = (y_train == '5') # 5는 True고, 다른 숫자는 모두 False
y_test_5 = (y_test == '5')
```

- SGDClassifier 모델을 만들고 전체 훈련 세트를 사용해 훈련
 - 확률적 경사 하강법(Stochastic Gradient Descent, SGD)

```
from sklearn.linear_model import SGDClassifier

sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train_5)
```



```
SGDClassifier
SGDClassifier(random_state=42)
```

- 숫자 5의 이미지를 감지

```
sgd_clf.predict([some_digit])
```



```
array([ True])
```


02 MNIST

■ 교차 검증을 사용한 정확도 측정

- cross_val_score() 함수
 - 폴드가 3개인 k-폴드 교차 검증을 사용해 SGDClassifier 모델을 평가

```
from sklearn.model_selection import cross_val_score
cross_val_score(sgd_clf, X_train, y_train_5, cv=3,
                 scoring="accuracy")
```

⇒ array([0.95035, 0.96035, 0.9604])

- 모든 이미지를 가장 많이 등장하는 클래스 (여기서는 음성 클래스, 즉 '5 아님')로 분류하는 더미 분류기를 만들어 비교

```
from sklearn.dummy import DummyClassifier

dummy_clf = DummyClassifier()
dummy_clf.fit(X_train, y_train_5)
print(any(dummy_clf.predict(X_train)))
```

⇒ False

- 모델의 정확도 확인

```
cross_val_score(dummy_clf, X_train, y_train_5, cv=3,
                 scoring="accuracy")
```

⇒ array([0.90965, 0.90965, 0.90965])

02 MNIST

■ 오차 행렬

- 오차 행렬 - 모든 A/B 쌍에 대해 클래스 A의 샘플이 클래스 B로 분류된 횟수를 세는 것
- cross_val_predict() 함수
 - 실제 타겟과 비교할 수 있도록 예측값 만들기
 - k-폴드 교차 검증을 수행하지만 평가 점수를 반환하지 않고 각 테스트 폴드에서 얻은 예측을 반환

```
from sklearn.model_selection import cross_val_predict

y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3)
```

02 MNIST

- confusion_matrix() 함수
 - 오차 행렬을 만들기

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_train_5, y_train_pred)
cm
```

```
array([[53892,  687],
       [1891, 3530]])
```

- 오차 행렬의 행은 실제 클래스, 열은 예측한 클래스를 나타냄
 - 예제 행렬의 첫 번째 행은 '5 아님' 이미지(음성 클래스 negative class)에 대한 것
 - 53,892개를 '5 아님'으로 정확하게 분류 - 진짜 음성(true negative)
 - 나머지 687개는 '5'라고 잘못 분류 - 거짓 양성(false positive) 또는 1종 오류(type I error)
 - 두 번째 행은 '5' 이미지(양성 클래스 positive class)에 대한 것
 - 1,891개를 '5 아님'으로 잘못 분류 - 거짓 음성(false negative) 또는 2종 오류(type II error)
 - 나머지 3,530개를 정확히 '5'라고 분류 - 진짜 양성(true positive)

02 MNIST

- 완벽한 분류기라면 진짜 양성과 진짜 음성만 가지고 있을 것이므로 오차 행렬의 주대각선(왼쪽 위에서 오른쪽 아래로)만 0이 아닌 값이 됨

```
y_train_perfect_predictions = y_train_5 # 완벽한 분류기일 경우
confusion_matrix(y_train_5, y_train_perfect_predictions)
```

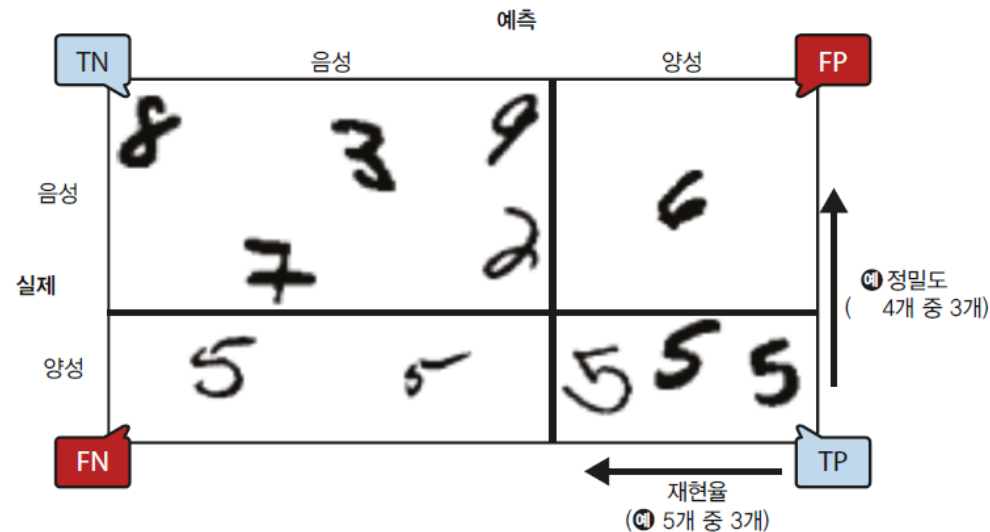
```
↔ array([[54579,    0],
        [    0, 5421]])
```

02 MNIST

- 분류기의 정밀도(precision) - 양성 예측의 정확도
 - TP는 진짜 양성의 수이고, FP는 거짓 양성의 수
- 정밀도는 재현율(recall)이라는 또 다른 지표와 같이 사용하는 것이 일반적
 - 재현율은 분류기가 정확하게 감지한 양성 샘플의 비율로, 민감도(sensitivity) 또는 진짜 양성 비율(true positive rate, TPR)
 - FN은 거짓 음성의 수

$$\text{정밀도} = \frac{TP}{TP + FP}$$

$$\text{재현율} = \frac{TP}{TP + FN}$$



오차 행렬 그림 - 진짜 음성 샘플(TN), 거짓 양성(FP), 거짓 음성(FN), 진짜 양성(TP)

02 MNIST

정밀도와 재현율

- 사이킷런은 정밀도와 재현율을 포함하여 분류기의 지표를 계산하는 여러 함수를 제공

```
from sklearn.metrics import precision_score, recall_score

precision_score(y_train_5, y_train_pred) # == 3530 / (687 + 3530)
```

⇒ 0.8370879772350012

```
recall_score(y_train_5, y_train_pred) # == 3530 / (1891 + 3530)
```

⇒ 0.6511713705958311

02 MNIST

- F_1 점수(F_1 score)는 정밀도와 재현율의 조화 평균(harmonic mean)

$$F_1 = \frac{2}{\frac{1}{\text{정밀도}} + \frac{1}{\text{재현율}}} = 2 \times \frac{\text{정밀도} \times \text{재현율}}{\text{정밀도} + \text{재현율}} = \frac{TP}{TP + \frac{FN + FP}{2}}$$

- `f1_score()` 함수 - F_1 점수를 계산

```
from sklearn.metrics import f1_score  
  
f1_score(y_train_5, y_train_pred)
```

⇒ 0.7325171197343846

02 MNIST

정밀도/재현율 트레이드오프

- SGDClassifier의 분류 결정 과정
 - 결정 함수(decision function)를 사용하여 각 샘플의 점수를 계산
 - 점수가 임계값보다 크면 샘플을 양성 클래스에 할당하고 그렇지 않으면 음성 클래스에 할당
 - 결정 임계값이 높으면 재현율이 낮아지고 정밀도가 올라감
 - 임계값을 내리면 재현율이 높아지고 정밀도가 줄어듦

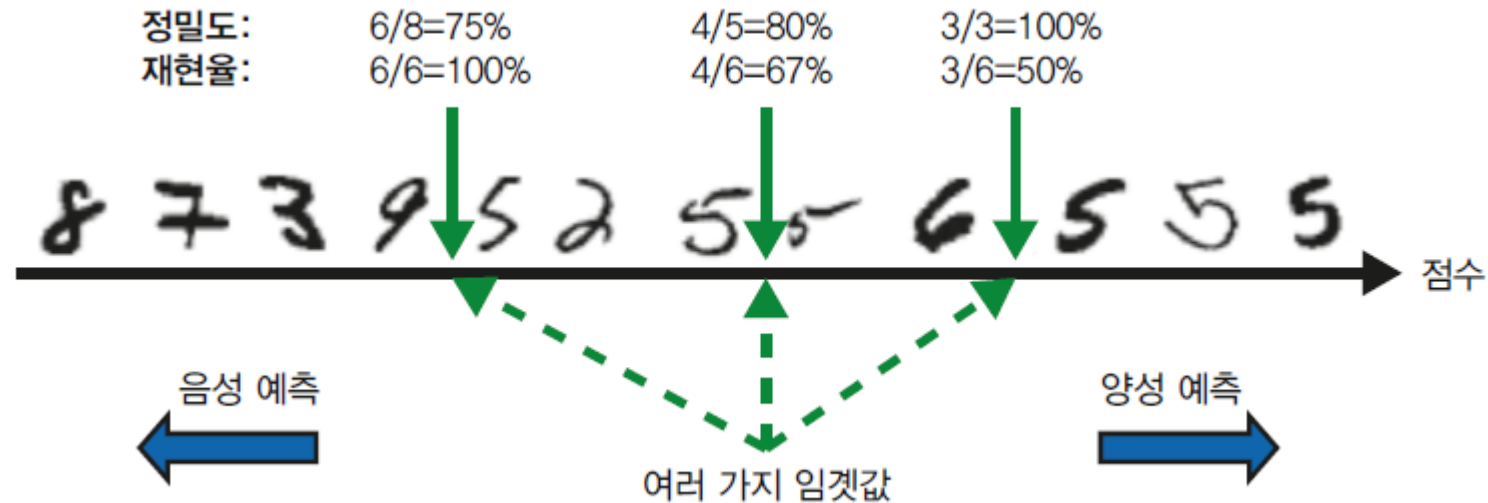


그림 3-4 이 정밀도/재현율 트레이드오프 이미지는 분류기가 만든 점수 순으로 나열. 선택한 결정 임계값 위에 있는 이미지를 양성으로 판단. 임계값이 높을수록 재현율은 낮아지고 반대로 (보통) 정밀도는 높아짐

02 MNIST

- decision_function() 메서드
 - 각 샘플의 점수를 확인하고 이 점수를 기반으로 원하는 임계값을 정해 예측 만들기
 - SGDClassifier의 임계값이 0이므로 predict() 메서드와 같은 결과(즉, True)를 반환

```
y_scores = sgd_clf.decision_function([some_digit])
y_scores
```

```
⇒ array([2164.22030239])
```

```
threshold = 0
y_some_digit_pred = (y_scores > threshold)
y_some_digit_pred
```

```
⇒ array([ True])
```

- 임계값을 높여서 재현율이 줄어드는 것을 확인

```
threshold = 3000
y_some_digit_pred = (y_scores > threshold)
y_some_digit_pred
```

```
⇒ array([False])
```

02 MNIST

■ 적절한 임계값 결정

- 먼저 `cross_val_predict()` 함수를 사용해 훈련 세트에 있는 모든 샘플의 점수를 계산
 - 이번에는 예측 결과가 아니라 결정 점수를 반환하도록 지정

```
y_scores = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3,
                             method="decision_function")
```

- 이 점수로 `precision_recall_curve()` 함수를 사용하여 가능한 모든 임계값에 대해 정밀도와 재현율을 계산

```
from sklearn.metrics import precision_recall_curve

precisions, recalls, thresholds = precision_recall_curve(y_train_5,
y_scores)
```

- 맷플롯립을 이용해 임계값의 함수로 정밀도와 재현율 시각화

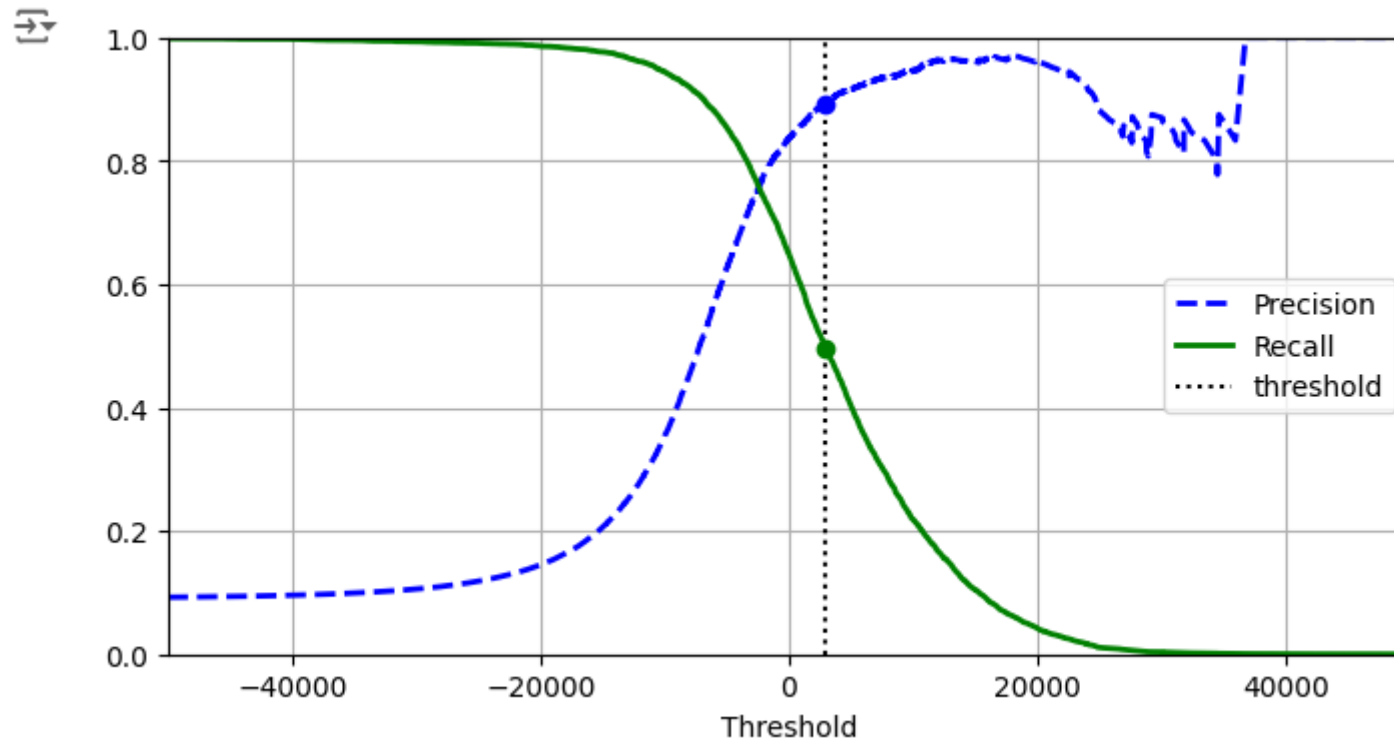
```
plt.figure(figsize=(8, 4)) # 추가 코드
plt.plot(thresholds, precisions[:-1], "b--", label="Precision", linewidth=2)
plt.plot(thresholds, recalls[:-1], "g-", label="Recall", linewidth=2)
plt.vlines(threshold, 0, 1.0, "k", "dotted", label="threshold")

# 추가 코드 - 그림 3-5를 그리고 저장합니다
idx = (thresholds >= threshold).argmax() # 첫 번째 index ≥ threshold
plt.plot(thresholds[idx], precisions[idx], "bo")
plt.plot(thresholds[idx], recalls[idx], "go")
plt.axis([-50000, 50000, 0, 1])
plt.grid()
plt.xlabel("Threshold")
plt.legend(loc="center right")

plt.show()
```

02 MNIST

- 맷플롯립을 이용해 임계값의 함수로 정밀도와 재현율 시각화



- 재현율에 대한 정밀도 곡선을 그려서 좋은 정밀도/재현율 트레이드오프를 선택

```
import matplotlib.patches as patches # 추가 코드 - 구부러진 화살표를 그리기 위해서

plt.figure(figsize=(6, 5)) # 추가 코드

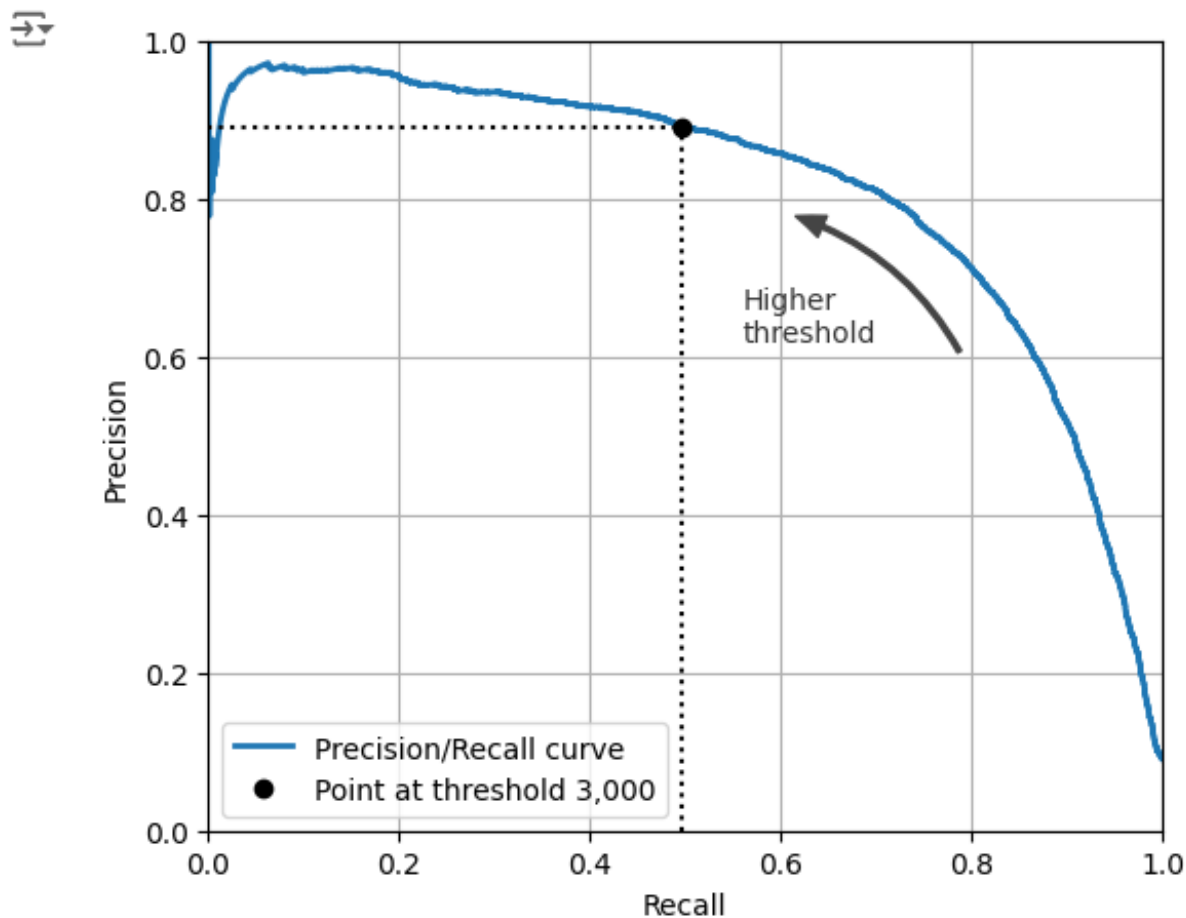
plt.plot(recalls, precisions, linewidth=2, label="Precision/Recall curve")

# extra code - just beautifies and saves Figure 3-6
plt.plot([recalls[idx], recalls[idx]], [0., precisions[idx]], "k:")
plt.plot([0.0, recalls[idx]], [precisions[idx], precisions[idx]], "k:")
plt.plot([recalls[idx], [precisions[idx]], "ko",
          label="Point at threshold 3,000")
plt.gca().add_patch(patches.FancyArrowPatch(
    (0.79, 0.60), (0.61, 0.78),
    connectionstyle="arc3,rad=.2",
    arrowstyle="Simple, tail_width=1.5, head_width=8, head_length=10",
    color="#444444"))
plt.text(0.56, 0.62, "Higher\ nthreshold", color="#333333")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.axis([0, 1, 0, 1])
plt.grid()
plt.legend(loc="lower left")

plt.show()
```

02 MNIST

- 재현율에 대한 정밀도 곡선을 그려서 좋은 정밀도/재현율 트레이드오프를 선택



- 넘파이 배열의 `argmax()` 메서드
 - 정밀도가 최소 90%가 되는 가장 낮은 임계값을 찾기

```
idx_for_90_precision = (precisions >= 0.90).argmax()  
threshold_for_90_precision = thresholds[idx_for_90_precision]  
threshold_for_90_precision
```

- (훈련 세트에 대한) 예측을 만들려면 분류기의 `predict()` 메서드를 호출하는 대신 다음 코드를 실행

```
y_train_pred_90 = (y_scores >= threshold_for_90_precision)
```

- 예측에 대한 정밀도와 재현율을 확인

```
precision_score(y_train_5, y_train_pred_90) ➡ 0.9000345901072293
```

```
recall_at_90_precision = recall_score(y_train_5,  
y_train_pred_90)  
recall_at_90_precision ➡ 0.4799852425751706
```

- 맷플롯립을 사용해 TPR에 대한 FPR 곡선 구현

```
idx_for_threshold_at_90 = (thresholds <= threshold_for_90_precision).argmax()
tpr_90, fpr_90 = tpr[idx_for_threshold_at_90], fpr[idx_for_threshold_at_90]

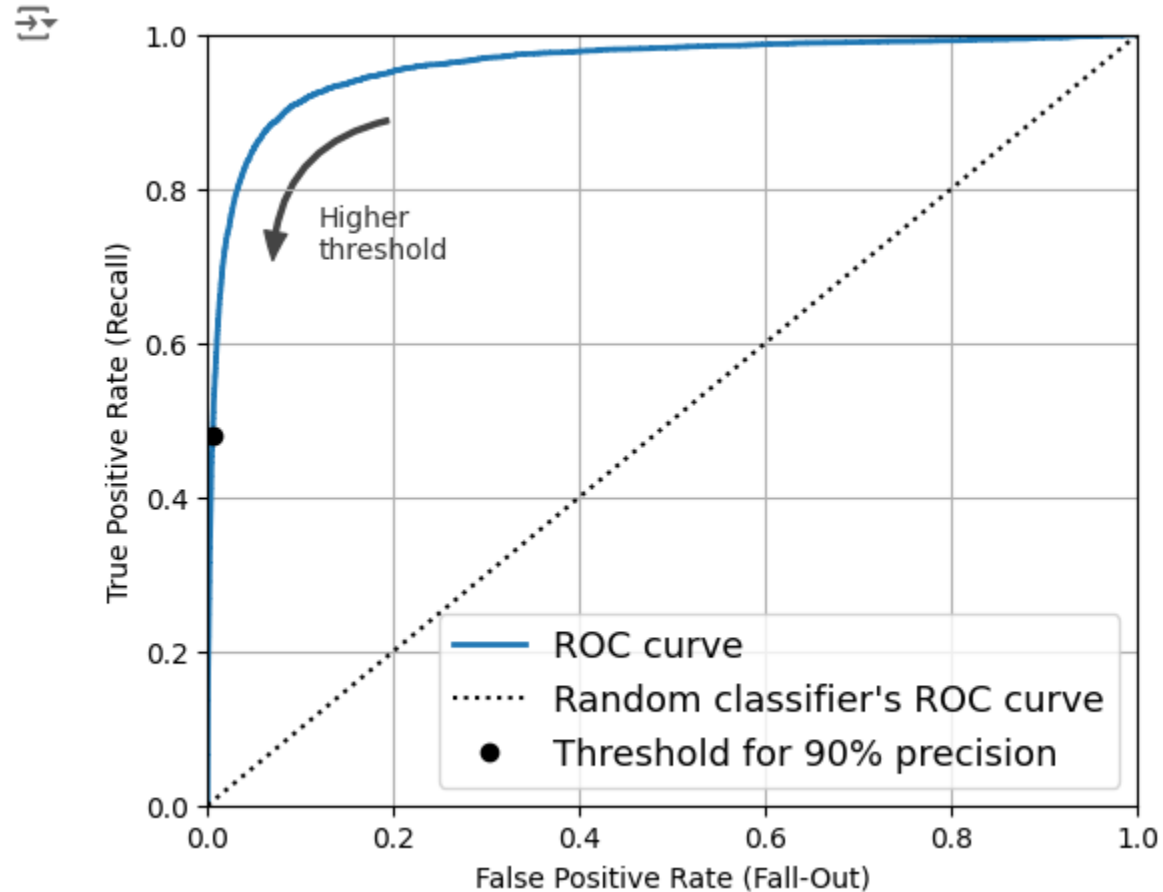
plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, linewidth=2, label="ROC curve")
plt.plot([0, 1], [0, 1], 'k:', label="Random classifier's ROC curve")
plt.plot([fpr_90], [tpr_90], "ko", label="Threshold for 90% precision")

# 그림 그리고 저장
plt.gca().add_patch(patches.FancyArrowPatch(
    (0.20, 0.89), (0.07, 0.70),
    connectionstyle="arc3,rad=.4",
    arrowstyle="Simple, tail_width=1.5, head_width=8, head_length=10",
    color="#444444"))

plt.text(0.12, 0.71, "Higher\ nthreshold", color="#333333")
plt.xlabel('False Positive Rate (Fall-Out)')
plt.ylabel('True Positive Rate (Recall)')
plt.grid()
plt.axis([0, 1, 0, 1])
plt.legend(loc="lower right", fontsize=13)

plt.show()
```


- 맷플롯립을 사용해 TPR에 대한 FPR 곡선 구현



가능한 모든 임계값에서 진짜 양성 비율에 대한 거짓 양성 비율을 나타낸 ROC 곡선
검은 점이 선택한 비율의 지점(90%의 정밀도와 48%의 재현율)

02 MNIST

- 곡선 아래의 면적(area under the curve, AUC)을 측정해 분류기들을 비교
 - 완벽한 분류기는 ROC의 AUC가 1이고, 완전한 랜덤 분류기는 0.5

```
from sklearn.metrics import roc_auc_score
```

```
roc_auc_score(y_train_5, y_scores)
```



0.9604938554008616

- RandomForestClassifier를 만들어 SGDClassifier의 PR 곡선과 F_1 점수를 비교

```
from sklearn.ensemble import RandomForestClassifier
```

```
forest_clf = RandomForestClassifier(random_state=42)
```

- cross_val_predict() 함수

- 교차 검증으로 RandomForestClassifier를 훈련하고 모든 이미지에 대한 클래스 확률을 예측

```
y_probas_forest = cross_val_predict(forest_clf, X_train, y_train_5, cv=3,  
                                   method="predict_proba")
```

- 훈련 세트에 있는 처음 두 개의 이미지에 대한 클래스 확률을 확인

```
y_probas_forest[:2]
```

```
⇒ array([[0.11, 0.89],  
         [0.99, 0.01]])
```

- 두 번째 열에 양수 클래스에 대한 추정 확률이 포함되어 있으므로 이를 precision_recall_curve() 함수에 전달

```
y_scores_forest = y_probas_forest[:, 1]  
precisions_forest, recalls_forest, thresholds_forest = precision_recall_curve(  
    y_train_5, y_scores_forest)
```

02 MNIST

- PR 곡선을 그려서 서로 비교

```
plt.figure(figsize=(6, 5)) # 추가 코드

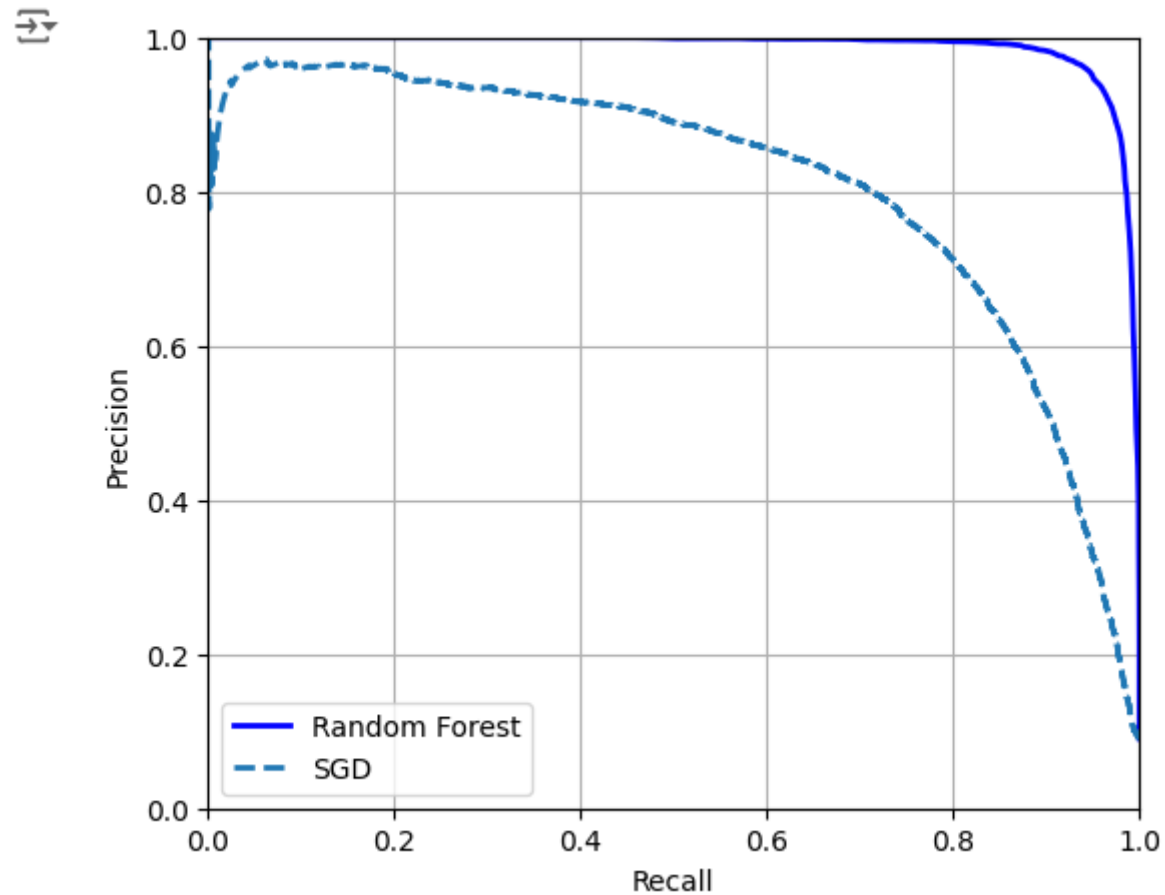
plt.plot(recalls_forest, precisions_forest, "b-", linewidth=2,
         label="Random Forest")
plt.plot(recalls, precisions, "--", linewidth=2, label="SGD")

plt.xlabel("Recall")
plt.ylabel("Precision")
plt.axis([0, 1, 0, 1])
plt.grid()
plt.legend(loc="lower left")

plt.show()
```

02 MNIST

- PR 곡선을 그려서 서로 비교



PR 곡선 비교: 랜덤 포레스트 분류기가 SGD 분류기보다 훨씬 좋음
랜덤 포레스트의 PR 곡선이 오른쪽 위 모서리에 훨씬 가까워 AUC가 더 높기 때문임

02 MNIST

- RandomForestClassifier의 PR 곡선이 SGDClassifier의 곡선보다 훨씬 더 좋아 보임
 - 즉, 오른쪽 위 모서리에 훨씬 더 가까움
- F_1 점수와 ROC AUC 점수도 훨씬 더 우수

```
y_train_pred_forest = y_probas_forest[:, 1] >= 0.5 # 양성 확률 ≥ 50%
f1_score(y_train_5, y_train_pred_forest)
```

⇒ 0.9274509803921569

```
roc_auc_score(y_train_5, y_scores_forest)
```

⇒ 0.9983436731328145

02 MNIST

- 다중 분류기(multiclass classifier) 또는 다항 분류기(multinomial classifier)는 둘 이상의 클래스를 구별
- OvR (one-versus-the-rest) 전략 또는 OvA(one-versus-all)
 - 이미지를 분류할 때 각 분류기의 결정 점수 중에서 가장 높은 것을 클래스로 선택
 - 대부분의 이진 분류 알고리즘에서는 OvR을 선호
- OvO (one-versus-one) 전략
 - 0과 1 구별, 0과 2 구별, 1과 2 구별 등과 같이 각 숫자의 조합마다 이진 분류기를 훈련
 - 장점 - 각 분류기의 훈련에 전체 훈련 세트 중 구별할 두 클래스에 해당하는 샘플만 있으면 됨
 - 작은 훈련 세트에서 많은 분류기를 훈련시키는 쪽이 빠르므로 OvO를 선호
- 다중 클래스 분류 작업에 이진 분류 알고리즘을 선택하면 사이킷런이 알고리즘에 따라 자동으로 OvR 또는 OvO를 실행

02 MNIST

- sklearn.svm.SVC 클래스를 사용해 서포트 벡터 머신 분류기를 테스트
 - 처음 2,000개의 이미지만 사용해 훈련

```
from sklearn.svm import SVC

svm_clf = SVC(random_state=42)
svm_clf.fit(X_train[:2000], y_train[:2000]) # y_train_5가 아니고 y_train 사용
```

- 이미지에 대한 예측 만들기

```
svm_clf.predict([some_digit])
```

```
array(['5'], dtype=object)
```

- decision_function() 메서드를 호출하면 샘플마다 총 10개의 점수(클래스마다 하나씩)를 반환

```
some_digit_scores = svm_clf.decision_function([some_digit])
some_digit_scores.round(2)
```

- 가장 높은 점수는 9.3이고 이는 클래스 5에 해당

```
class_id = some_digit_scores.argmax()
class_id
```

```
5
```


- 분류기가 훈련될 때 `classes_` 속성에 타깃 클래스의 리스트를 값으로 정렬하여 저장
 - 클래스 레이블 확인

```
svm_clf.classes_
```

```
array(['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'], dtype=object)
```

```
svm_clf.classes_[class_id]
```

```
'5'
```

- OvO나 OvR을 사용하도록 강제하려면 `OneVsOneClassifier`나 `OneVsRestClassifier`를 사용
 - SVC 기반으로 OvR 전략을 사용하는 다중 분류기

```
from sklearn.multiclass import OneVsRestClassifier  
  
ovr_clf = OneVsRestClassifier(SVC(random_state=42))  
ovr_clf.fit(X_train[:2000], y_train[:2000])
```

- 예측을 만들고 훈련된 분류기 개수를 확인

```
ovr_clf.predict([some_digit])
```

```
array(['5'], dtype='<U1')
```

```
len(ovr_clf.estimators_)
```

```
10
```

- 다중 분류 데이터셋에서 SGDClassifier를 훈련하고 예측

※주의:실행하는데 오래 걸릴 수 있음.

```
sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train)
sgd_clf.predict([some_digit])
```

⇒ array(['3'], dtype='<U1')

- SGD 분류기가 각 클래스에 부여한 점수를 확인

```
sgd_clf.decision_function([some_digit]).round()
```

⇒ array([[-31893., -34420., -9531., 1824., -22320., -1386., -26189.,
 -16148., -4604., -12051.]])

- cross_val_score() 함수를 사용해 이 모델을 평가

```
cross_val_score(sgd_clf, X_train, y_train, cv=3, scoring="accuracy")
```

⇒ array([0.87365, 0.85835, 0.8689])

- 입력의 스케일을 조정하여 정확도를 89.1% 이상으로 높임

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train.astype("float64"))
cross_val_score(sgd_clf, X_train_scaled, y_train, cv=3,
scoring="accuracy")
```

⇒ array([0.8983, 0.891 , 0.9018])

02 MNIST

- 모델의 성능 향상을 위한 방법 - 생성된 오류의 종류를 분석
- 오차 행렬을 관찰
 - cross_val_predict() 함수를 사용해 예측을 만들고 confusion_matrix() 함수를 호출
- 오차 행렬을 컬러 그래프로 시각화 분석
 - ConfusionMatrixDisplay.from_predictions() 함수를 사용

```
from sklearn.metrics import ConfusionMatrixDisplay

y_train_pred = cross_val_predict(sgd_clf, X_train_scaled, y_train, cv=3)
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred)
plt.show()
```

- 오차 행렬을 정규화
 - normalize="true"로 지정하면 이 작업을 간단히 수행
 - values_format=".0%" 매개변수를 지정하여 소수점 없이 백분율을 표시

```
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred,
                                       normalize="true", values_format=".0%")
plt.show()
```

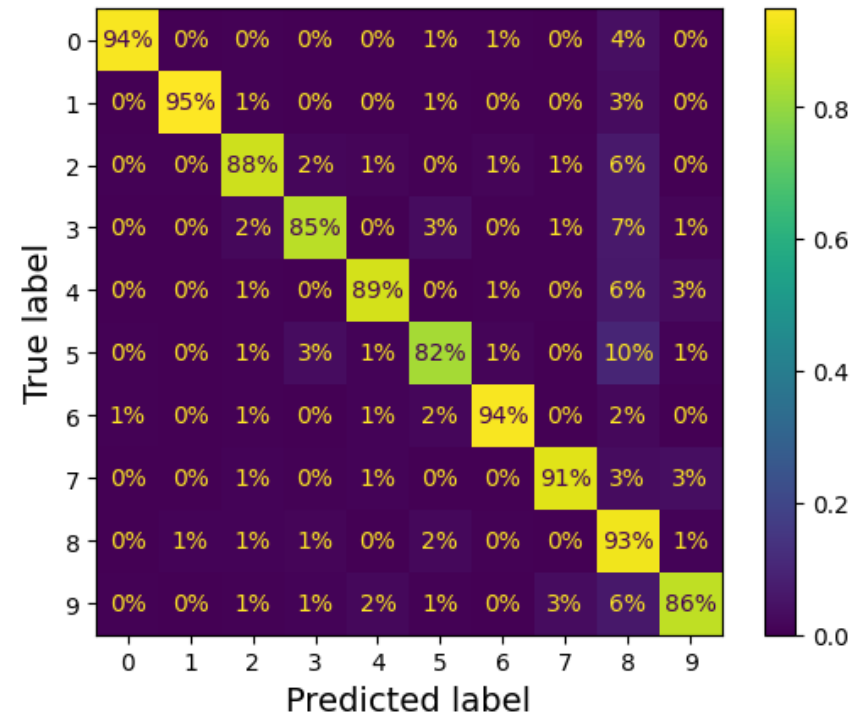
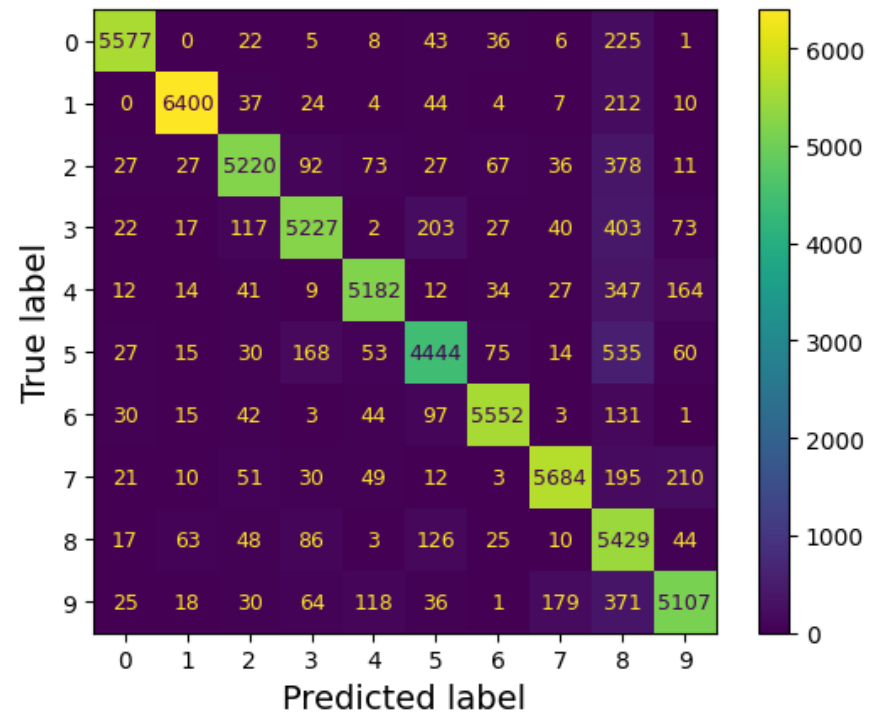


그림 3-9 오차 행렬(왼쪽)과 행별로 정규화된 오차 행렬(오른쪽)

02 MNIST

- 오류를 더 눈에 띄게 만들기 위해 올바른 예측에 대한 가중치를 0으로 설정
 - 오차 행렬을 행 단위가 아닌 열 단위로 정규화하려면 `normalize="pred"`로 지정

```
sample_weight = (y_train_pred != y_train)
plt.rc('font', size=10) # 추가 코드
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred,
                                       sample_weight=sample_weight,
                                       normalize="true", values_format=".0%")
plt.show()
```

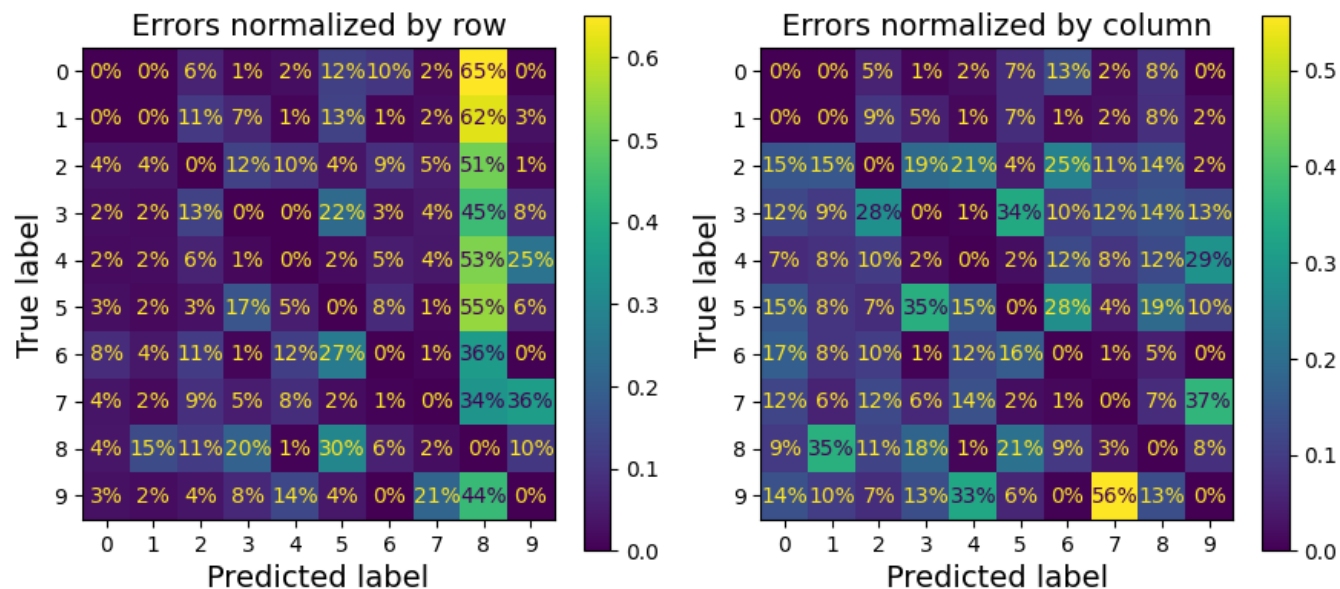
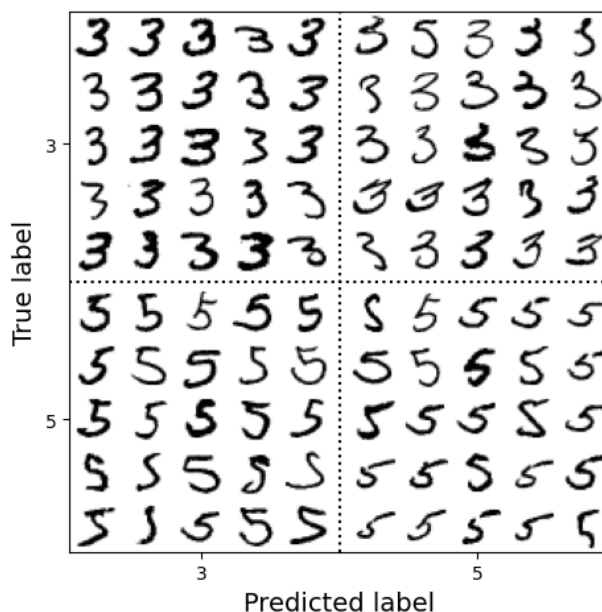


그림 3-10 행별로 정규화된 오차 행렬(왼쪽)과 열별로 정규화된 오차 행렬(오른쪽)

02 MNIST

- 오차 행렬을 분석하면 분류기의 성능 향상 방안에 관한 인사이트를 얻음
 - 오차 행렬 스타일로 그린 3과 5의 샘플

```
cl_a, cl_b = '3', '5'  
X_aa = X_train[(y_train == cl_a) & (y_train_pred == cl_a)]  
X_ab = X_train[(y_train == cl_a) & (y_train_pred == cl_b)]  
X_ba = X_train[(y_train == cl_b) & (y_train_pred == cl_a)]  
X_bb = X_train[(y_train == cl_b) & (y_train_pred == cl_b)]
```



오차 행렬 스타일로 그린 3과 5의 일부 이미지

02 MNIST

- 선형 분류기는 클래스마다 픽셀에 가중치를 할당하고 새로운 이미지에 대해 단순히 픽셀 강도의 가중치 합을 클래스의 점수로 계산
 - 몇 개의 픽셀만 다른 3과 5를 모델이 쉽게 혼동
- 3과 5의 주요 차이는 위쪽 선과 아래쪽 호를 이어주는 작은 직선의 위치
 - 숫자 3을 쓸 때 연결 부위가 조금 왼쪽으로 치우지면 분류기가 5로 분류하고 그 반대도 마찬가지
 - 분류기는 이미지의 위치나 회전 방향에 매우 민감
 - 3과 5의 오류를 줄이는 한 가지 방법은 이미지를 중앙에 위치시키고 회전되어 있지 않도록 전처리
- 데이터 증식(data augmentation)
 - 훈련 이미지를 약간 이동시키거나 회전된 변형 이미지로 훈련 집합을 보강

02 MNIST

■ 다중 레이블 분류(multilabel classification) 시스템

- 여러 개의 이진 꼬리표를 출력하는 분류 시스템
- 각 숫자 이미지에 두 개의 타깃 레이블이 담긴 y_multilabel 배열 만들기
 - 첫 번째는 숫자가 큰 값(7, 8, 9)인지 나타내고 두 번째는 홀수인지 나타냄
 - 그다음 코드는 KNeighborsClassifier 인스턴스를 만들고 다중 타깃 배열을 사용하여 훈련

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

y_train_large = (y_train >= '7')
y_train_odd = (y_train.astype('int8') % 2 == 1)
y_multilabel = np.c_[y_train_large, y_train_odd]

knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train, y_multilabel)
```

- 예측을 만들면 레이블이 두 개 출력

```
knn_clf.predict([some_digit])          array([[False,  True]])
```


02 MNIST

- 다중 레이블 분류기 평가
 - 각 레이블의 F_1 점수를 구하고(또는 앞서 언급한 어떤 이진 분류 지표를 사용하여) 간단하게 평균 점수를 계산
 - 이 코드는 모든 레이블의 가중치가 같다고 가정

```
y_train_knn_pred = cross_val_predict(knn_clf, X_train, y_multilabel, cv=3)
f1_score(y_multilabel, y_train_knn_pred, average="macro")
```

- 레이블에 클래스의 지지도(support, 타깃 레이블에 속한 샘플 수)를 가중치로 주기
 - f1_score() 함수를 호출할 때 average="weighted"로 설정

- SVC와 같이 기본적으로 다중 레이블 분류를 지원하지 않는 분류기를 사용하는 경우
 - 레이블당 하나의 모델을 학습
 - 이 전략은 레이블 간의 의존성을 포착하기 어렵게 할 수 있음
 - 문제를 해결하기 위해 모델을 체인(chain)으로 구성
 - 한 모델이 예측을 할 때 입력 특성과 체인 앞에 있는 모델의 모든 예측을 사용
 - ClassifierChain 클래스는 훈련에 진짜 레이블을 사용하며 체인 내 위치에 따라 각 모델에 적절한 레이블을 공급

02 MNIST

- 교차 검증 전략을 사용하여 ClassifierChain을 만들고 훈련하는 방법
 - 속도를 높이기 위해 훈련 세트에서 처음 2,000개 이미지만 사용

```
from sklearn.multiooutput import ClassifierChain

chain_clf = ClassifierChain(SVC(), cv=3, random_state=42)
chain_clf.fit(X_train[:2000], y_multilabel[:2000])
```

- ClassifierChain을 사용해 예측

```
chain_clf.predict([some_digit])

array([[0., 1.]])
```

02 MNIST

- 다중 출력 다중 클래스 분류(multioutput-multiclass classification), 다중 출력 분류(multioutput classification)
 - 다중 레이블 분류에서 한 레이블이 다중 클래스가 될 수 있도록 일반화(즉, 값을 두 개 이상 가질 수 있음)
- (학습 예제) 이미지에서 잡음을 제거하는 시스템
 - 잡음이 많은 숫자 이미지를 입력으로 받아, 깨끗한 숫자 이미지를 MNIST 이미지처럼 픽셀의 강도를 담은 배열로 출력
 - MNIST 이미지에서 추출한 훈련 세트와 테스트 세트에 넘파이의 randint() 함수를 사용하여 픽셀 강도에 잡음을 추가

```
np.random.seed(42) # 동일하게 재현되게 하려고 지정합니다
noise = np.random.randint(0, 100, (len(X_train), 784))
X_train_mod = X_train + noise
noise = np.random.randint(0, 100, (len(X_test), 784))
X_test_mod = X_test + noise
y_train_mod = X_train
y_test_mod = X_test
```

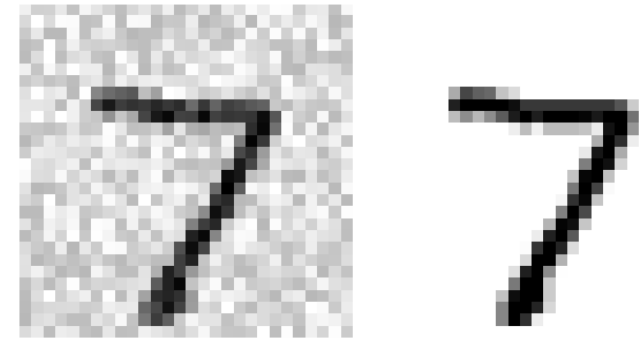


그림 3-12 잡음이 섞인 이미지(왼쪽)와 깨끗한 타겟 이미지(오른쪽)

02 MNIST

- 다중 출력 다중 클래스 분류(multioutput-multiclass classification), 다중 출력 분류(multioutput classification)
 - 다중 레이블 분류에서 한 레이블이 다중 클래스가 될 수 있도록 일반화(즉, 값을 두 개 이상 가질 수 있음)
- (학습 예제) 이미지에서 잡음을 제거하는 시스템
 - 잡음이 많은 숫자 이미지를 입력으로 받아, 깨끗한 숫자 이미지를 MNIST 이미지처럼 픽셀의 강도를 담은 배열로 출력
 - MNIST 이미지에서 추출한 훈련 세트와 테스트 세트에 넘파이의 randint() 함수를 사용하여 픽셀 강도에 잡음을 추가

```
knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train_mod, y_train_mod)
clean_digit = knn_clf.predict([X_test_mod[0]])
plot_digit(clean_digit)
plt.show()
```



그림 3-13 깨끗하게 정리된 이미지

Q & A